Noctalia

You are viewing the **v4 (stable)** documentation. Switch to v5 (alpha) →

Panel

Panels are full-screen overlay components that provide detailed interfaces for your plugin. They can be opened from bar widgets or triggered by other events.

Basic Structure

A panel is a QML file with this structure:

```
import QtQuick
import QtQuick.Layouts
```



```
import qs.Widgets

Item {
  id: root

  // Plugin API (injected by PluginPanelSlot)
  property var pluginApi: null

  // SmartPanel properties (required for panel behavior)
  readonly property var geometryPlaceholder: panelContainer
  readonly property bool allowAttach: true

  // Preferred dimensions
  property real contentPreferredWidth: 680 * Style.uiScaleRatio
  property real contentPreferredHeight: 540 * Style.uiScaleRatio

  anchors.fill: parent

  Rectangle {
    id: panelContainer
    anchors.fill: parent
    color: "transparent"

    // Your panel content here
  }
}
```

Required Properties

pluginApi

Injected by the PluginPanelSlot when the panel is loaded.

```
property var pluginApi: null
```

SmartPanel Properties


```
// Container for sizing calculations
readonly property var geometryPlaceholder: panelContainer

// Allow panel to attach to screen edges
readonly property bool allowAttach: true
```

Panel Dimensions

Set preferred dimensions for your panel:

```
property real contentPreferredWidth: 680 * Style.uiScaleRatio
property real contentPreferredHeight: 540 * Style.uiScaleRatio
```

> **Tip**
>
> Multiply by Style.uiScaleRatio to respect user's UI scaling preferences.

Panel Content

Organize panel content with layouts:

```
Rectangle {
  id: panelContainer
  color: "transparent"

  ColumnLayout {
    anchors {
      fill: parent
      margins: Style.marginL
    }
    spacing: Style.marginL

    // Header
    NText {
      text: "Panel Title"
      pointSize: Style.fontSizeL
      font.weight: Font.Bold
      color: Color.mOnSurface
    }

    // Content area
    Rectangle {
      Layout.fillWidth: true
      Layout.fillHeight: true
      color: Color.mSurfaceVariant
      radius: Style.radiusL

      // Your content here
    }

    // Footer or actions
    RowLayout {
      Layout.fillWidth: true
      spacing: Style.marginM

      NButton {
        text: "Action"
        onClicked: {
          // Handle action
```

```
    }
  }
```



```
}
```

Opening the Panel

Panels are typically opened from bar widgets:

```
// In BarWidget.qml
MouseArea {
  anchors.fill: parent
  onClicked: {
    if (pluginApi) {
      pluginApi.openPanel(root.screen, root)
    }
  }
}
```

Or programmatically:

```
function showDetails(buttonItem) {
  pluginApi.openPanel(root.screen, buttonItem)
}
```

> **Tip**
>
> Passing a button/widget reference as the second parameter makes the panel open near that widget.

Closing the Panel

Users can close panels by:

- Clicking outside the panel
- Pressing Escape

To close programmatically:

```
NButton {
  text: "Close"
  onClicked: {
    pluginApi.closePanel(pluginApi.panelOpenScreen)
  }
}
```

Using Settings

Access and modify plugin settings in panels:

```
ColumnLayout {
  spacing: Style.marginM
```



```
  // Display current setting
  NText {
    text: "Current: " + (pluginApi?.pluginSettings?.message || "")
    color: Color.mOnSurface
  }

  // Modify setting
  NTextInput {
    id: messageInput
    Layout.fillWidth: true
    label: "Message"
    text: pluginApi?.pluginSettings?.message || ""
  }

  NButton {
    text: "Save"
    onClicked: {
      pluginApi.pluginSettings.message = messageInput.text
      pluginApi.saveSettings()
      ToastService.showNotice("Settings saved!")
    }
  }
}
```

Styling

Use Noctalia's design system for consistent styling:

Background Colors

```
Rectangle {
  // Main surface
```



```
  // Variant surface (for cards, containers)
  color: Color.mSurfaceVariant

  // Transparent (inherit from panel background)
  color: "transparent"
}
```

Text Colors

```
NText {
  // Primary text
  color: Color.mOnSurface

  // Secondary text
  color: Color.mOnSurfaceVariant

  // Accent text
  color: Color.mPrimary
}
```

Spacing and Layout

```
ColumnLayout {
  anchors {
    margins: Style.marginL // Outer margins
  }
  spacing: Style.marginL // Between sections

  RowLayout {
    spacing: Style.marginM // Between related items
  }

  // For tight spacing
  spacing: Style.marginS
  spacing: Style.marginXS
}
```


Complete Examples

Simple Information Panel

```
import QtQuick
import QtQuick.Layouts
import qs.Widgets

Item {
  id: root

  property var pluginApi: null

  readonly property var geometryPlaceholder: panelContainer
  readonly property bool allowAttach: true

  property real contentPreferredWidth: 500 * Style.uiScaleRatio
  property real contentPreferredHeight: 400 * Style.uiScaleRatio

  anchors.fill: parent

  Rectangle {
    id: panelContainer
    anchors.fill: parent
    color: "transparent"

    ColumnLayout {
      anchors {
        fill: parent
        margins: Style.marginL
      }
      spacing: Style.marginL

      // Header
      NText {
        text: "Plugin Information"
        pointSize: Style.fontSizeXL
        font.weight: Font.Bold
        color: Color.mOnSurface
      }

      // Content
```

```
    Rectangle {
      Layout.fillWidth: true
```



```
      radius: Style.radiusL

      ColumnLayout {
        anchors {
          fill: parent
          margins: Style.marginL
        }
        spacing: Style.marginM

        NIcon {
          Layout.alignment: Qt.AlignHCenter
          icon: "noctalia"
          pointSize: Style.fontSizeXXL * 2
        }

        NText {
          Layout.alignment: Qt.AlignHCenter
          text: pluginApi?.manifest?.name || "Plugin"
          pointSize: Style.fontSizeL
          font.weight: Font.Medium
          color: Color.mPrimary
        }

        NText {
          Layout.alignment: Qt.AlignHCenter
          text: pluginApi?.manifest?.description || ""
          pointSize: Style.fontSizeM
          color: Color.mOnSurfaceVariant
          horizontalAlignment: Text.AlignHCenter
          wrapMode: Text.WordWrap
          Layout.fillWidth: true
        }
      }
    }
  }
}
```

```
}
```


Interactive Settings Panel

```
import QtQuick
import QtQuick.Layouts
import qs.Widgets
import qs.Services.UI

Item {
  id: root

  property var pluginApi: null

  readonly property var geometryPlaceholder: panelContainer
  readonly property bool allowAttach: true

  property real contentPreferredWidth: 680 * Style.uiScaleRatio
  property real contentPreferredHeight: 540 * Style.uiScaleRatio

  anchors.fill: parent

  // Local state for editing
  property string editMessage: pluginApi?.pluginSettings?.message || ""
  property color editBgColor: pluginApi?.pluginSettings?.backgroundColor || "#A9AEFE"

  Rectangle {
    id: panelContainer
    anchors.fill: parent
    color: "transparent"

    ColumnLayout {
      anchors {
        fill: parent
        margins: Style.marginL
      }
      spacing: Style.marginL

      // Header
      RowLayout {
        Layout.fillWidth: true
```

```
    NText {
      text: "Configure Plugin"
```



```
      color: Color.mOnSurface
      Layout.fillWidth: true
    }

    NIconButton {
      icon: "x"
      onClicked: {
        pluginApi.closePanel(pluginApi.panelOpenScreen)
      }
    }
  }

  // Content
  Rectangle {
    Layout.fillWidth: true
    Layout.fillHeight: true
    color: Color.mSurfaceVariant
    radius: Style.radiusL

    ColumnLayout {
      anchors {
        fill: parent
        margins: Style.marginL
      }
      spacing: Style.marginM

      // Message input
      NTextInput {
        Layout.fillWidth: true
        label: "Display Message"
        description: "The message shown in the bar widget"
        text: root.editMessage
        onTextChanged: root.editMessage = text
      }

      // Color picker
```

```
ColumnLayout {
  Layout.fillWidth: true
```



```
  NLabel {
    label: "Background Color"
    description: "Widget background color"
  }

  NColorPicker {
    Layout.preferredWidth: Style.sliderWidth
    Layout.preferredHeight: Style.baseWidgetSize
    selectedColor: root.editBgColor
    onColorSelected: function(color) {
      root.editBgColor = color
    }
  }
}

// Preview
NText {
  text: "Preview:"
  pointSize: Style.fontSizeM
  font.weight: Font.Medium
  color: Color.mOnSurface
  Layout.topMargin: Style.marginM
}

Rectangle {
  Layout.fillWidth: true
  Layout.preferredHeight: 60
  color: root.editBgColor
  radius: Style.radiusM

  NText {
    anchors.centerIn: parent
    text: root.editMessage
    pointSize: Style.fontSizeM
    color: Color.mOnPrimary
  }
```

```
    }
  }
```



```
  // Actions
  RowLayout {
    Layout.fillWidth: true
    spacing: Style.marginM

    Item {
      Layout.fillWidth: true
    }

    NButton {
      text: "Cancel"
      onClicked: {
        // Reset to saved values
        root.editMessage = pluginApi?.pluginSettings?.message || ""
        root.editBgColor = pluginApi?.pluginSettings?.backgroundColor || "#A9AEFE"
        pluginApi.closePanel(pluginApi.panelOpenScreen)
      }
    }

    NButton {
      text: "Save"
      highlighted: true
      onClicked: {
        // Save settings
        pluginApi.pluginSettings.message = root.editMessage
        pluginApi.pluginSettings.backgroundColor = root.editBgColor.toString()
        pluginApi.saveSettings()

        ToastService.showNotice("Settings saved!")
        pluginApi.closePanel(pluginApi.panelOpenScreen)
      }
    }
  }
}
```

```
}
```


List Panel With Actions

```
import QtQuick
import QtQuick.Layouts
import qs.Widgets

Item {
  id: root

  property var pluginApi: null

  readonly property var geometryPlaceholder: panelContainer
  readonly property bool allowAttach: true

  property real contentPreferredWidth: 600 * Style.uiScaleRatio
  property real contentPreferredHeight: 500 * Style.uiScaleRatio

  anchors.fill: parent

  ListModel {
    id: itemsModel
    ListElement { title: "Item 1"; description: "First item" }
    ListElement { title: "Item 2"; description: "Second item" }
    ListElement { title: "Item 3"; description: "Third item" }
  }

  Rectangle {
    id: panelContainer
    anchors.fill: parent
    color: "transparent"

    ColumnLayout {
      anchors {
        fill: parent
        margins: Style.marginL
      }
      spacing: Style.marginL

      // Header with search
      ColumnLayout {
```

```
    Layout.fillWidth: true
    spacing: Style.marginM
```



```
    text: "Items List"
    pointSize: Style.fontSizeL
    font.weight: Font.Bold
    color: Color.mOnSurface
   }

   NTextInput {
    id: searchInput
    Layout.fillWidth: true
    placeholderText: "Search items..."
    icon: "search"
   }
  }

  // List
  Rectangle {
   Layout.fillWidth: true
   Layout.fillHeight: true
   color: Color.mSurfaceVariant
   radius: Style.radiusL

   NScrollView {
    anchors.fill: parent

    ListView {
     model: itemsModel
     spacing: Style.marginS

     delegate: Rectangle {
      width: ListView.view.width - Style.marginL * 2
      height: 60
      x: Style.marginL
      color: Color.mSurface
      radius: Style.radiusM

      RowLayout {
```

```
    anchors {
      fill: parent
```



```
    spacing: Style.marginM

    ColumnLayout {
      Layout.fillWidth: true
      spacing: Style.marginXS

      NText {
        text: model.title
        pointSize: Style.fontSizeM
        font.weight: Font.Medium
        color: Color.mOnSurface
      }

      NText {
        text: model.description
        pointSize: Style.fontSizeS
        color: Color.mOnSurfaceVariant
      }
    }

    NIconButton {
      icon: "dots-vertical"
      onClicked: {
        Logger.i("Panel", "Action for:", model.title)
      }
    }
  }

  MouseArea {
    anchors.fill: parent
    hoverEnabled: true
    onEntered: parent.color = Qt.lighter(Color.mSurface, 1.1)
    onExited: parent.color = Color.mSurface
    onClicked: {
      Logger.i("Panel", "Selected:", model.title)
    }
```

```
        }
      }
```



```
    }
  }
 }
}
```

Best Practices

1. **Use proper dimensions**: Set realistic contentPreferredWidth and contentPreferredHeight
2. **Respect UI scale**: Multiply dimensions by Style.uiScaleRatio
3. **Use margins**: Apply consistent margins with Style.margin*
4. **Provide feedback**: Show toasts or visual feedback for actions
5. **Handle empty states**: Show helpful messages when there's no data
6. **Support scrolling**: Use NScrollView for long content
7. **Test responsiveness**: Ensure panel works at different sizes
8. **Clean up**: Properly close panels after actions complete

Accessing Panel from Other Components

From Bar Widget

BarWidget.qml

```
MouseArea {
  anchors.fill: parent
  onClicked: pluginApi.openPanel(root.screen, root)
}
```

From Main.qml (IPC)

Main.qml


```
  function showPanel() {
    // Need to get screen reference asynchronously
    pluginApi.withCurrentScreen(screen => {
      pluginApi.openPanel(screen);
    });
  }
}
```

See Also

- Bar Widget Development - Create bar widgets
- Desktop Widget Development - Create desktop widgets
- Plugin API - Full API reference
- Getting Started - Plugin development basics
- Manifest Reference - Plugin configuration